

CPS305 Lab 3

Topics:

- BST

Files:

- Lab3.py
- Incorrectly named files will receive a mark of zero.

Submit Files:

- Submit file through D2L
- Your file should not have any print statements in it.
- You will lose a mark for each print statement.
- There is a `getStudentNumber()` function in the Lab3.py file.
- Change the output string to student number.
- You MUST have your student number filled out to receive the mark for the lab.

Lab Description:

In this lab, you are implementing a linked solution of your own BST. You are given Python class templates of the `MyBST` class. Some of the functionalities are given and you are to complete the rest as indicated by the requirements.

Requirements:

- First, create a method called `getStudentNumber()` that returns your TMU student number. If this does not work, you will receive a mark of zero.
- A class called `MyBST` is given. Write the method `remove()` to meet the definitions of a Binary Search Tree. To properly implement the `remove()` method, you will also need to implement the methods `updateHeight()` and `findSmallest()` or `findLargest()`. The method `updateHeight()` updates the height of the node. Depending on which implementation you choose, you can use `findSmallest()` or `findLargest()` which find the smallest or largest nodes from the left subtree or right subtree of the removed node. Write the method called `__contains__(x)` to return true if x is in the tree.
- The following methods: `getLeft()`, `getRight()`, `getData()`, and `insert()` are given. Do not change them. Although the `insert()` method is given, it utilizes the `updateHeight()` method so that must be implemented correctly.
- The method `__init__()` is also given. You may add to this. Note the class variable `"promote_right."` Set it to true if you plan on implementing the convention that promotes from the right subtree when removing or set it to false otherwise.
- Your implementation of `updateHeight()`, `findSmallest()` or `findLargest()`, `remove()`, `__contains__()` **MUST** be implemented using recursion.

Restrictions:

- You cannot use Python's list in your implementation. This includes the methods `append`, `pop`, etc. This lab requires linked implementations of BST.

Testing Your Code:

You can test your code with the tester. The tester is not a be all and end all. You should rely on your own testing before using the tester. Getting 100% on the tester does not guarantee you 100% of the lab marks. You would lose marks if we can find scenario where your implementation fails.

Note that the lab should be done on Python 3 and the tester only works on Python 3.

The tester is named Lab3-Tester.py. Your file must be named correctly and be in the same folder of the tester. Here is an example of executing it on the command line (note that the \$ is representative of the beginning of the new line).

```
$ python3 Lab3-Tester.py
```